BUSCA INFORMADA

Prof. Wagner Igarashi, Dr.

Busca informada (heurística)

- Porque todas as estratégias anteriores são chamadas de desinformadas?
  - Porque elas não consideram qualquer informação sobre os estados para decidir qual caminho expandir em primeiro lugar na fronteira.
- Em outras palavras, elas são gerais e não levam em conta a natureza específica do problema.

Busca informada (heurística)

- **Ideia**: não ignore o objetivo ao selecionar os caminhos.
- Muitas vezes há conhecimento extra que pode ser usado para orientar a busca: uma **estimativa** da distância do nó ***n*** para um nó objetivo, por exemplo.
- Uma **heurística**, ***h(n)***, é uma estimativa do custo do caminho mais curto do nó ***n*** para um nó objetivo.
  - ***h*(*n*)** usa apenas informações que podem ser facilmente obtidas (que são fáceis de calcular) sobre um nó.
  - ***h*** pode ser estendido para caminhos: $h(\langle n_0, \ldots, n_k \rangle) = h(n_k)$.
  - ***h*(*n*)** é uma subavaliação se não houver nenhum caminho de ***n*** para um nó objetivo que tenha comprimento inferior a ***h*(*n*)**.

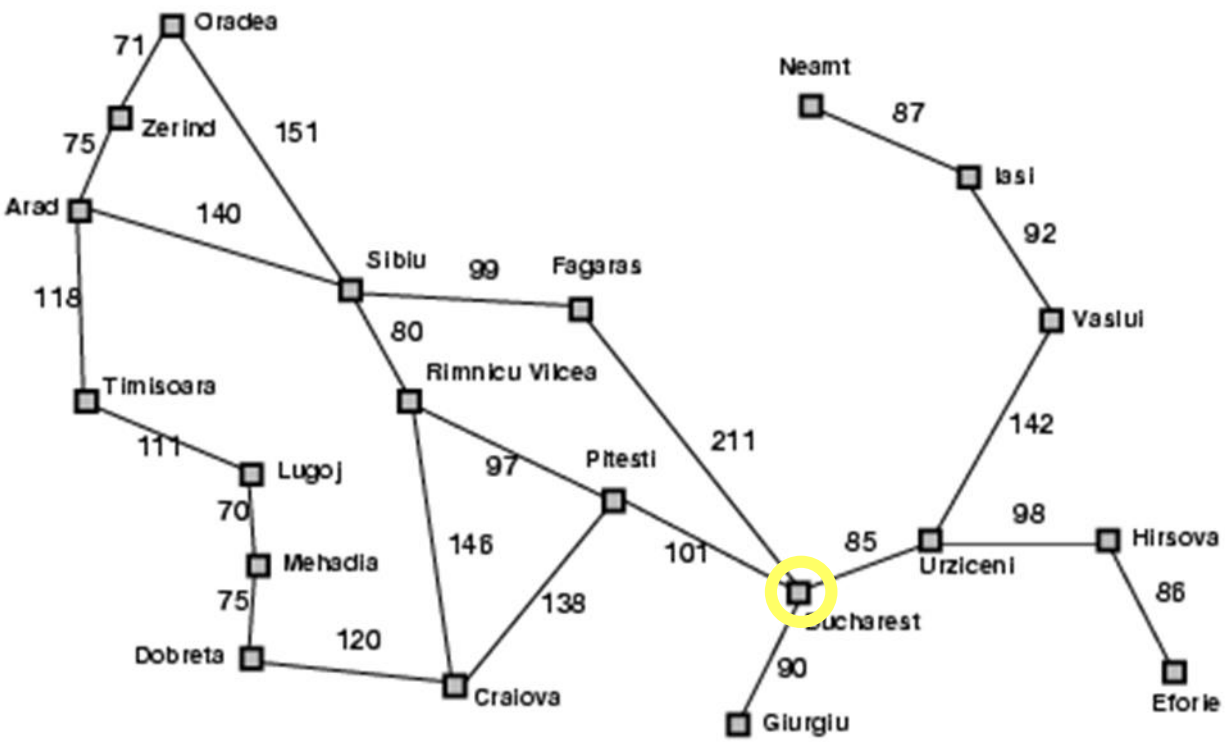
Funções Heurísticas

- Definição de **função heurística admissível:**
  - Uma função heurística ***h(n)*** é admissível se ela nunca é uma sobrestimação do custo do nó ***n*** para um nó objetivo.
- As heurísticas admissíveis tem natureza otimista, pois elas sempre indicam que o custo da solução é melhor do que ele realmente é.
  - Desta maneira se uma solução ainda não foi encontrada sempre existirá um nó com *custo* menor do que ela.

- Nunca existe um caminho do nó ***n*** para um nó objetivo que tenha caminho de comprimento inferior a ***h(n)***. Outra maneira de dizer isto:
  - ***h(n)*** é um limite inferior sobre o custo de ir do nó ***n*** para o nó objetivo mais próximo.

Exemplo de funções heurísticas admissíveis

- Se os nós são pontos em um plano Euclidiano e o custo é a distância, então podemos usar a distância linear entre o nó ***n*** e o nó objetivo mais próximo como o valor da ***h(n)***.

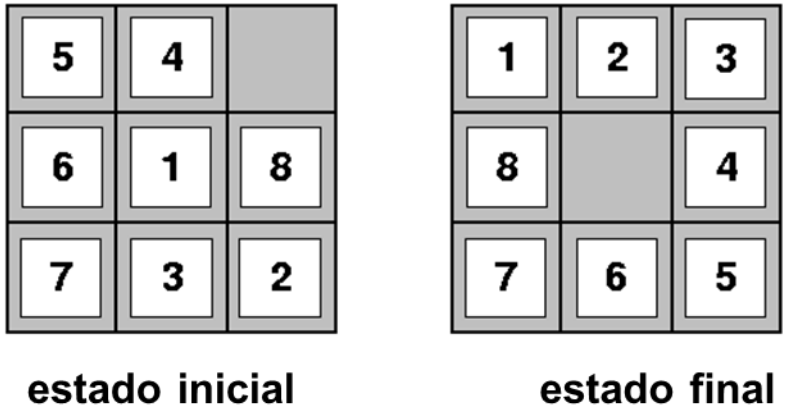


Straight–line distance to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

straight line distance = distância em linha reta

Exemplo de funções heurísticas admissíveis



- Exemplo de heurísticas para o quebra-cabeças de 8 peças
  - $h_1(n)$ = número de quadrados em locais errados
  - $h_2(n)$ = distância Manhattan total
    - número de espaços na vertical e horizontal em que as peças devem ser movidas para chegar no local correto (as peças podem se mover umas sobre as outras).
  - $h_1(s) = 7$ (só o número 7 está no local correto)
  - $h_2(s) = 2+3+3+2+4+2+0+2 = 18$

Como construir uma heurística

- Você deve identificar uma versão relaxada do problema:
  - Na qual uma ou mais restrições sobre as ações foram descartadas.

- **Exemplo**:
  - Robô: o agente pode mover-se através das paredes.
  - Motorista: o agente pode mover-se em linha reta ao local.
  - *8-puzzle*:
    - 1) os elementos podem mover para qualquer lugar.
    - 2) os elementos podem mover para qualquer casa adjacente.

- **Resultado**:
  - O custo de uma solução ideal para o problema relaxado é uma heurística admissível para o problema original (porque sempre é fracamente menos custoso resolver um problema menos restrito!)

Como construir uma heurística

- Você deve identificar restrições que, quando removidas, fazem o problema extremamente fácil de ser resolvido.
  - Isso é importante porque as heurísticas não são úteis se elas são tão difíceis de resolver quanto o problema original!
- Este é o caso em nossos exemplos:
  - Robô: ***permitindo*** que o agente se mova através das paredes, a solução ideal para este problema relaxado é a **distância Manhattan.**
  - Motorista: ***permitindo*** que o agente se mova em linha reta ao local, a solução ideal para este problema relaxado é **distância em linha reta**.
  - 8-*puzzle*: (1) se os elementos podem mover-se para qualquer lugar, a solução ideal para esse problema relaxado é o **número de peças no local errado**; (2) se os elementos podem mover-se para qualquer local adjacente ...

Heurística: dominância

- Se $h_2(n) \geq h_1(n)$ para todos os ***n*** (sendo ambas admissíveis) então h_2 domina h_1.
  - Qual é a melhor para a busca (porque?)

- **8-*puzzle***:
  - (1) os elementos podem mover-se para qualquer lugar.
  - (2) os elementos podem mover-se para qualquer casa adjacente.

- No problema original os elementos podem mover-se para um quadrado adjacentes se ele estiver vazio.

- Custos da busca para o 8-*puzzle* (número médio de caminhos expandidos):

d = 12	IDS = 3,644,035 caminhos	onde *d* é a profundidade da solução
	$A*(h_1)$ = 227 caminhos	
	$A*(h_2)$ = 73 caminhos	
d = 24	IDS = caminhos demais	
	$A*(h_1)$ = 39,135 caminhos	
	$A*(h_2)$ = 1,641 caminhos	

Heurística: dominância

- Se $h_2(n) \geq h_1(n)$ para todos os ***n*** (sendo ambas admissíveis) então h_2 domina h_1.
  - h_2 é melhor para a pesquisa

- **8-*puzzle*:**
  - (1) os elementos podem mover-se para qualquer lugar.
  - (2) os elementos podem mover-se para qualquer casa adjacente.

- No problema original os elementos podem mover-se para um quadrado adjacentes se ele estiver vazio.

- Custos da busca para o 8-*puzzle* (número médio de caminhos expandidos):

d = 12	IDS = 3,644,035 caminhos	onde *d* é a profundidade da solução
	$A*(h_1)$ = 227 caminhos	
	$A*(h_2)$ = 73 caminhos	
d = 24	IDS = caminhos demais	
	$A*(h_1)$ = 39,135 caminhos	
	$A*(h_2)$ = 1,641 caminhos	

Combinando heurísticas

- **Como combinar heurística quando não há nenhuma dominância**?

- Se $h_1(n)$ é admissível e $h_2(n)$ também é admissível, então $h(n) = \max(h_1, h_2)$ também é admissível e domina todos seus componentes.

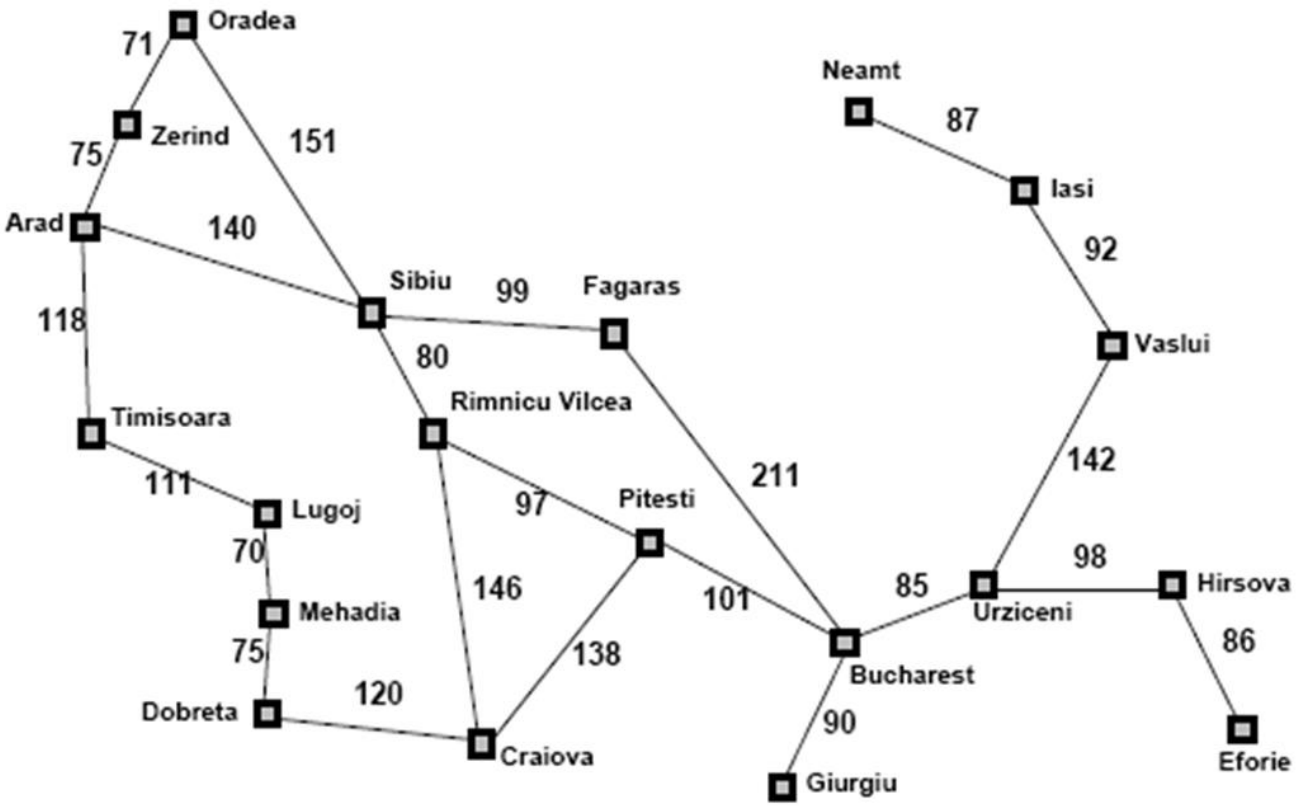
Busca heurística sistemática

- **Busca informada**
  - Busca pelo melhor-primeiro (*greedy* – gulosa)
  - Busca A*
  - Busca recursiva de melhor escolha – **anteriormente chamada de busca em aprofundamento por enumeração e poda**

Busca pelo melhor-primeiro (*greedy*)

- **Ideia**: selecione o caminho cujo fim está mais próximo a um nó objetivo de acordo com a função heurística.
- A busca pelo melhor-primeiro seleciona um caminho na fronteira com o valor ***h*** mínimo.
- Ela trata a fronteira como uma ***fila de prioridade*** ordenada por ***h*(*n*)**.
- É um algoritmo guloso que seleciona o melhor caminho local.

Gráfico ilustrativo – busca pelo melhor-primeiro

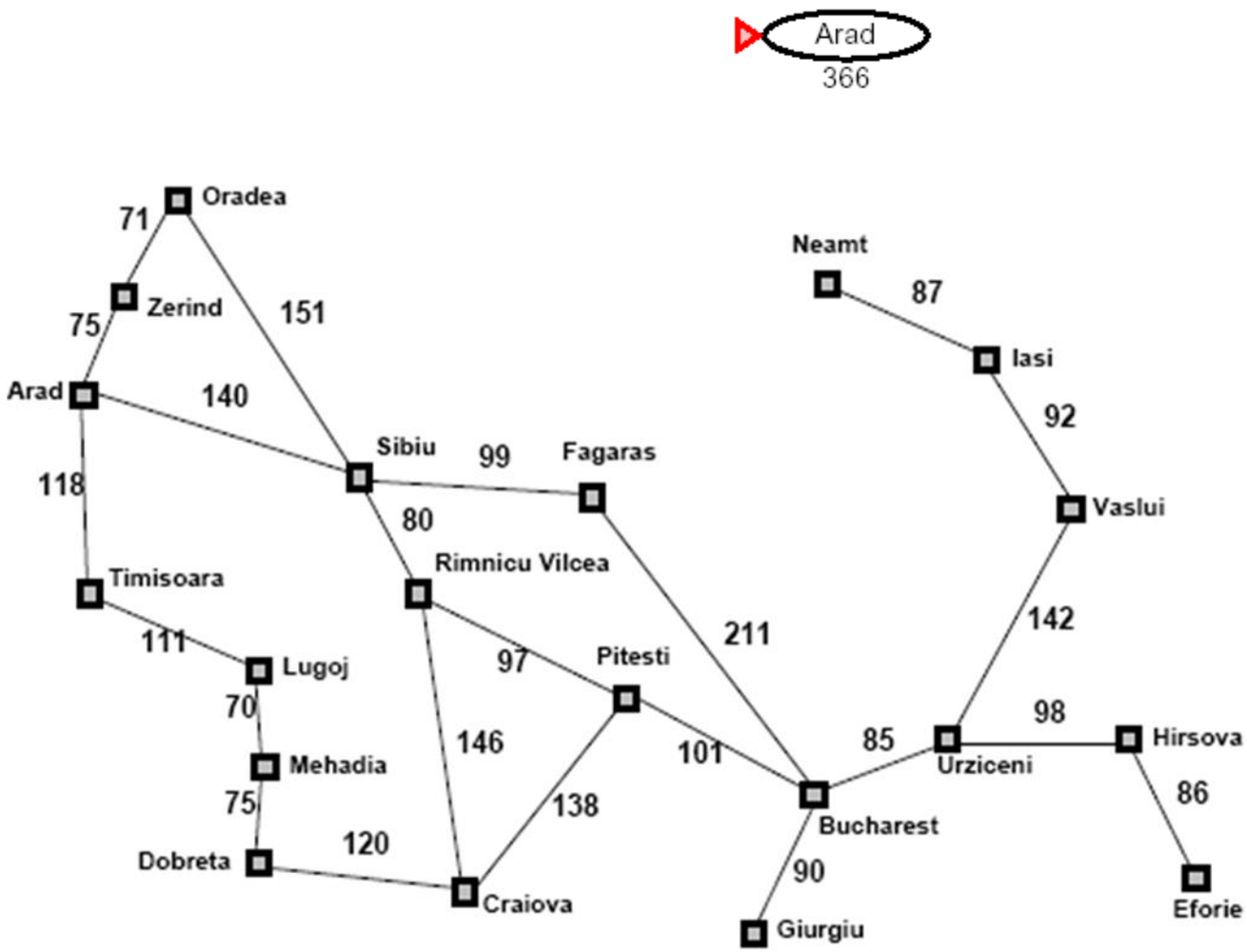


Straight−line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Exemplo: busca pelo melhor primeiro (gulosa)

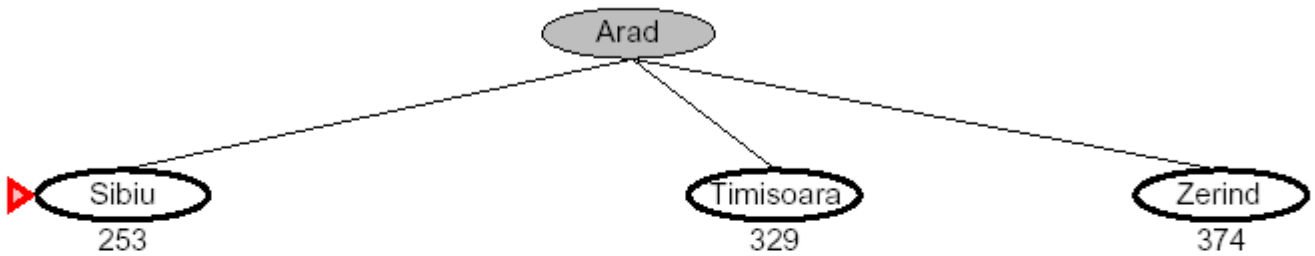
- ***Estado Inicial***: Em(Arad) = Arad
- ***Estado Final***: Em(Bucharest) = Bucharest
- ***Obs***.: Nós que podem gerar ciclos são marcados com *, e não são inseridos na fronteira
- **heurística**: onde estiver, pegar a cidade vizinha de menor distância (em linha reta) para cidade destino

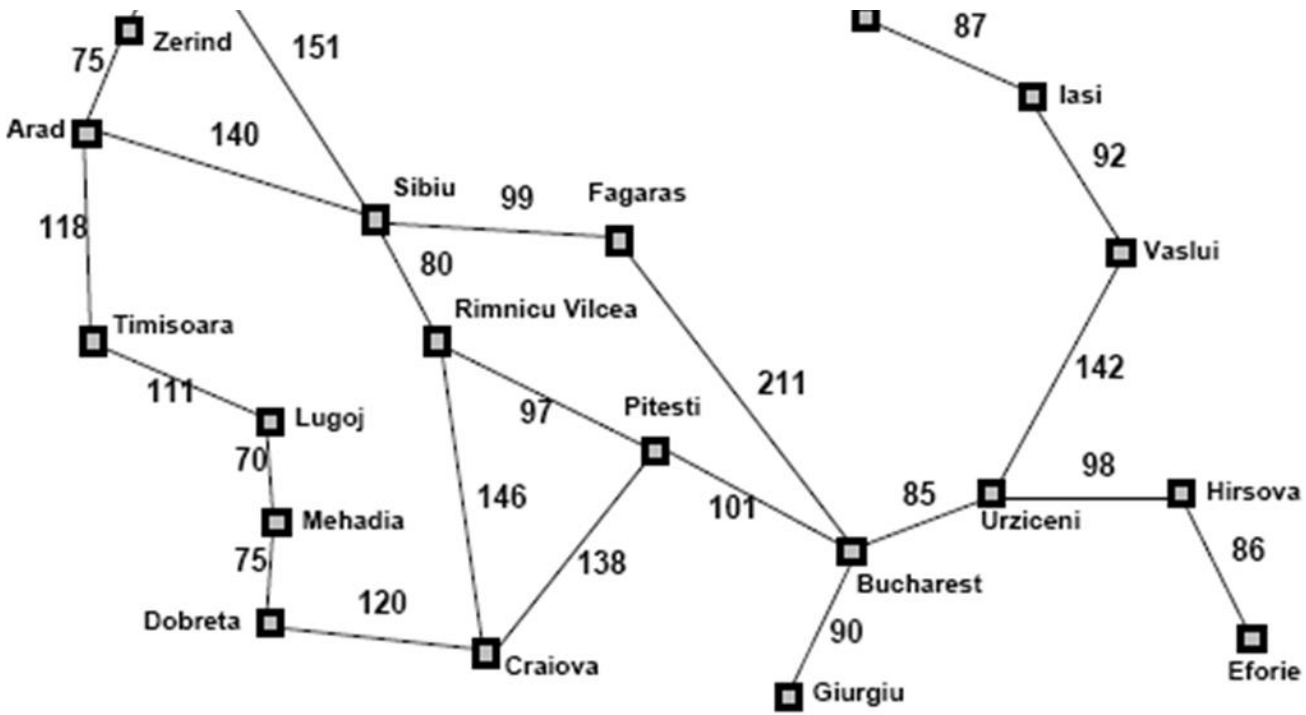
Exemplo: busca pelo melhor primeiro (gulosa)



Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Exemplo: busca pelo melhor primeiro (gulosa)



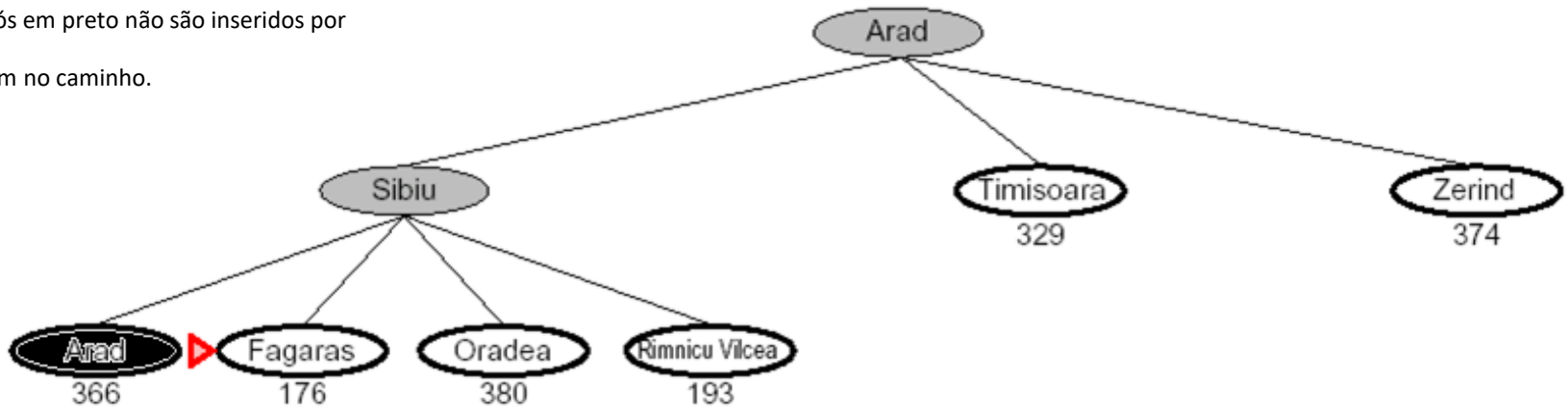


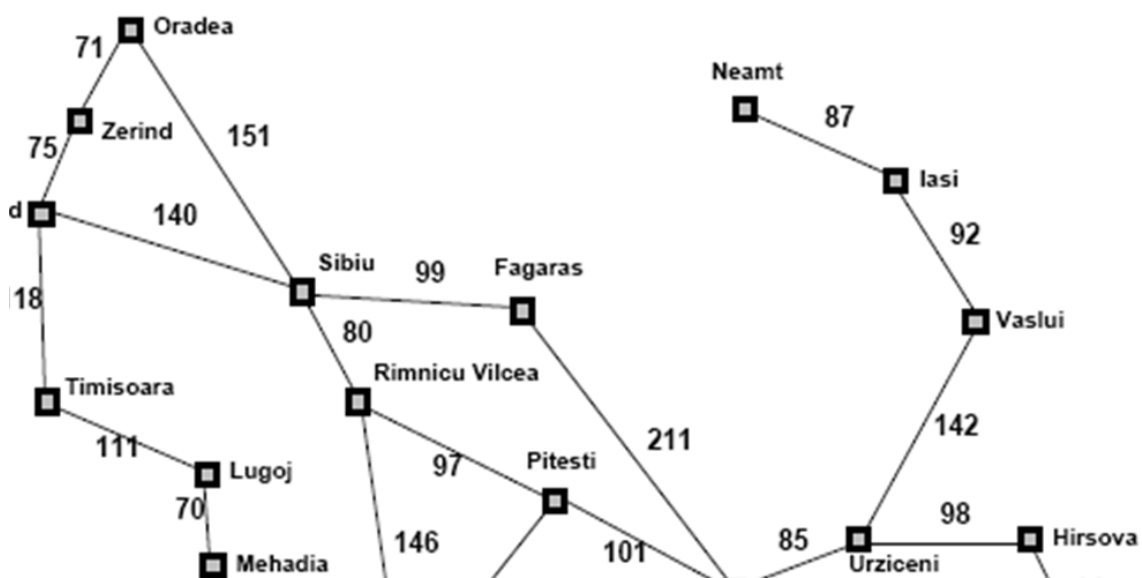
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Exemplo: busca pelo melhor primeiro (gulosa)

- Nós em cinza já foram expandidos
- Nós em preto não são inseridos por

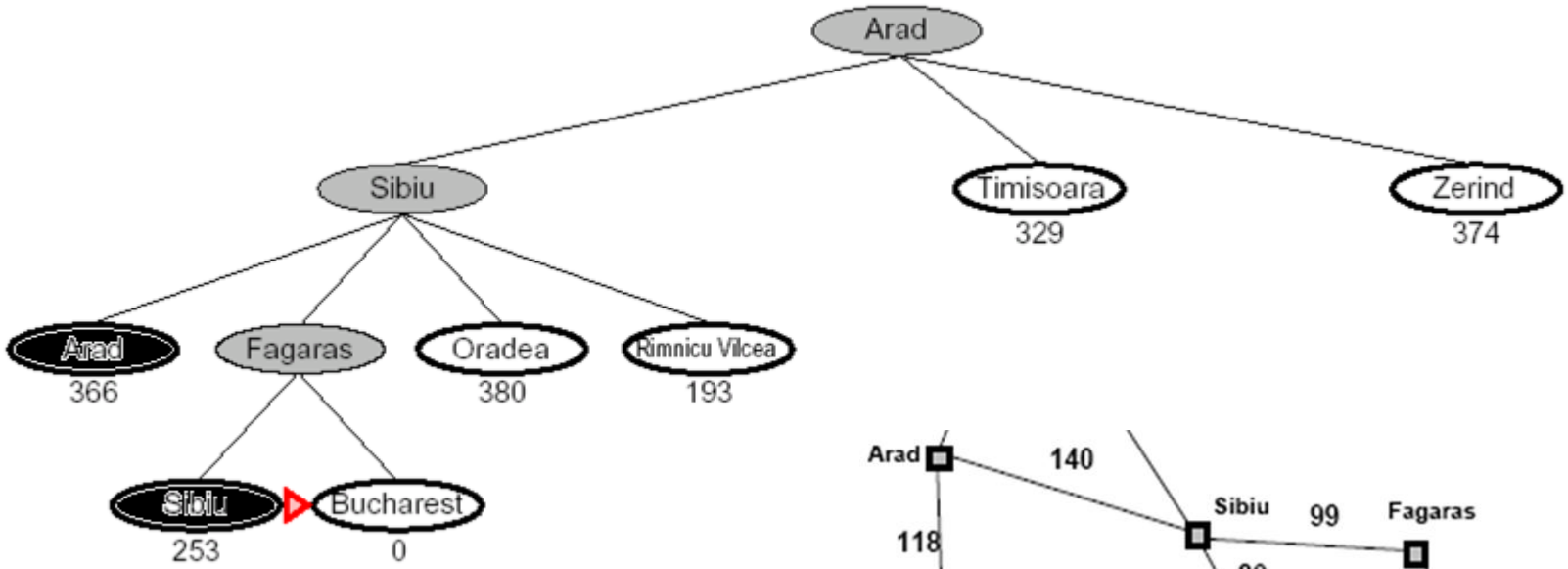
já estarem no caminho.

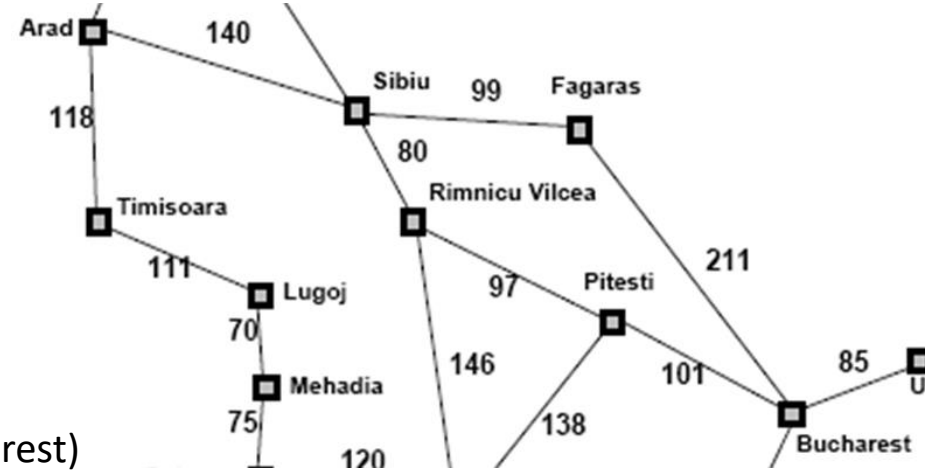




to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193

Exemplo: busca pelo melhor primeiro (gulosa)





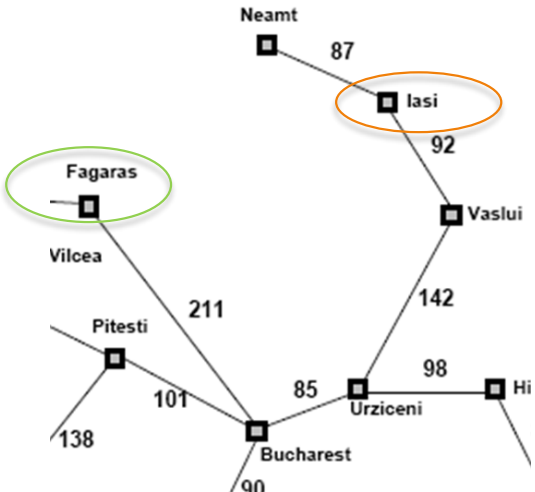
- O solução encontrada não é a solução ótima:
  140 + 99 + 211 = 450 (Arad > Sibiu > Fagaras > Bucharest)
- A solução ótima tem 32 km a menos:
  140 + 80 + 97 + 101 = 418 (Arad > Sibiu > Rimnicu Vilcea > Pitest > Bucharest)

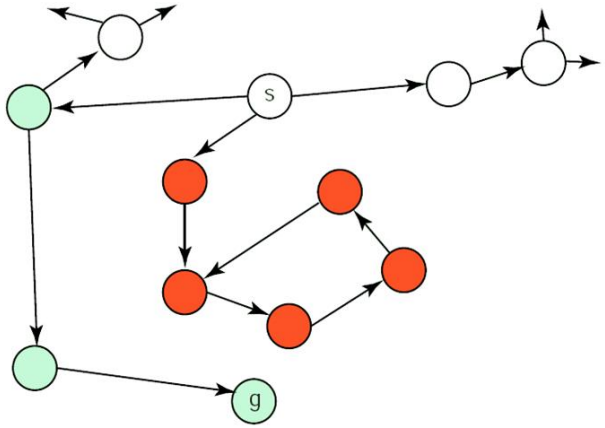
Busca pelo melhor primeiro (gulosa)


```
def busca_pelo_melhor_primeiro(G, S, goal):
   entrada:  G # a grafo
             S # um conjunto de nós iniciais
             objetivo # função booleana que testa se S é um nó objetivo

   saída: um caminho de um membro de S para um nó para o qual a função
   objetivo é verdadeira, ou ⊥ se não existir solução.

   Local: fronteira # um conjunto de caminhos


    fronteira ← {<s>|s ∈ S}
    while fronteira :
        selecione e remova o 1º nó <s_0,…,s_k> da fronteira

        if objetivo(s_k): return <s_0,…,s_k>

        adicione(ordem crescente de h(n), fronteira, {<s_0,…,s_k,s>}|<s_k,s> ∈ S})
    return ⊥
```

Complexidade do busca pelo melhor-primeiro

- Usa espaço exponencial no comprimento do caminho $O(\boldsymbol{b}^m)$.
- Usa tempo exponencial no comprimento do cominho $O(\boldsymbol{b}^m)$.
- Incompleta - não é garantido que ela encontra uma solução, mesmo se houver uma, pois um valor de heurística baixo pode fazer com que a busca fique em um ciclo para sempre.
  - No exemplo abaixo os nós laranja tem valor heurístico melhores que os azuis, os quais são o caminho certo para a solução
  - Ex: Ir de Iasi para Fagaras.
- Nem sempre encontra o caminho mais curto.





- Como melhorar?
  - Somar a distância de cada cidade vizinha com a sua menor distância em linha reta até a cidade destino.

Busca A*

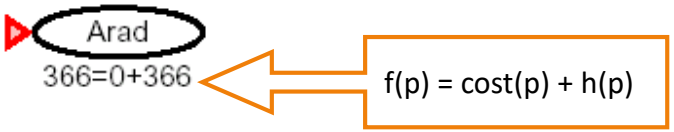
- Usa tanto o custo do caminho até o nó atual quanto o valor da heurística do nó atual.
- ***cost*(*p*)** é o custo do caminho até ***p***.
- ***h*(*p*)** estima o custo do final do caminho de ***p*** até um nó objetivo.
- Logo, ***f*(*p*) = *cost*(*p*) + *h*(*p*).**
  - ***f*(*p*)** estima o custo total do caminho para ir de um nó de início até um nó objetivo passando por ***p***.

$$\underbrace{\underbrace{start \xrightarrow{\text{path } p} n}_{cost(p)} \underbrace{\xrightarrow{\text{estimate}} goal}_{h(p)}}_{f(p)}$$

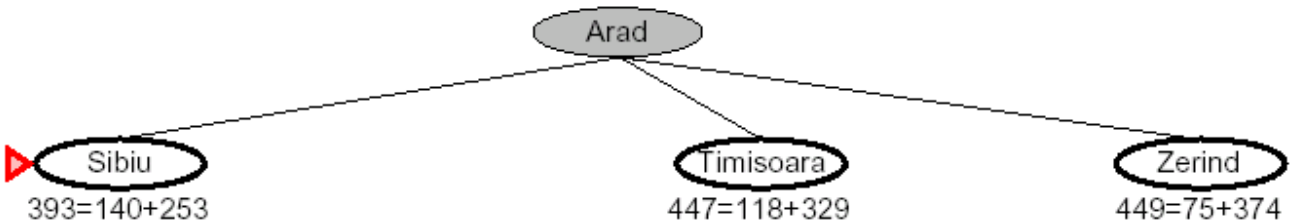
Exemplo: busca A*

- Problema:
  - Ir de **Arad** → **Bucharest**.
- Função heurística:
  - **Distância em linha reta** entre a cidade *n* e Bucharest.
  - Satisfaz a condição de admissibilidade, pois não existe distância menor entre dois pontos do que uma reta.
  - É uma boa heurística, pois induz o algoritmo a atingir o objetivo mais rapidamente.

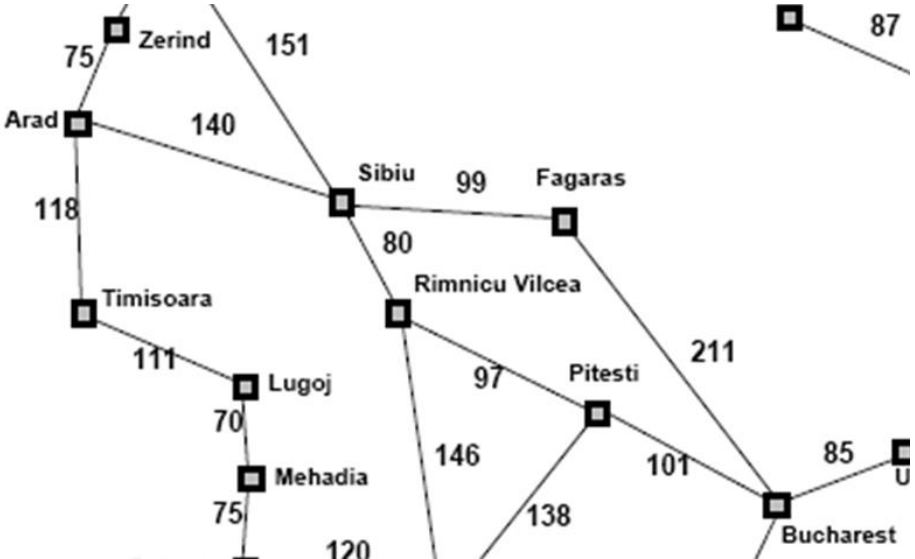
Exemplo: busca A*



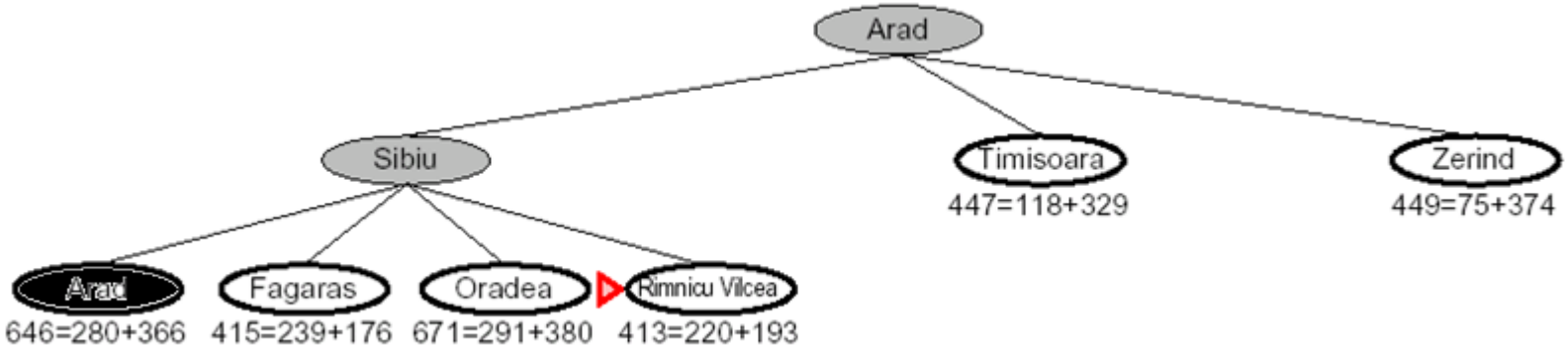
Exemplo: busca A*



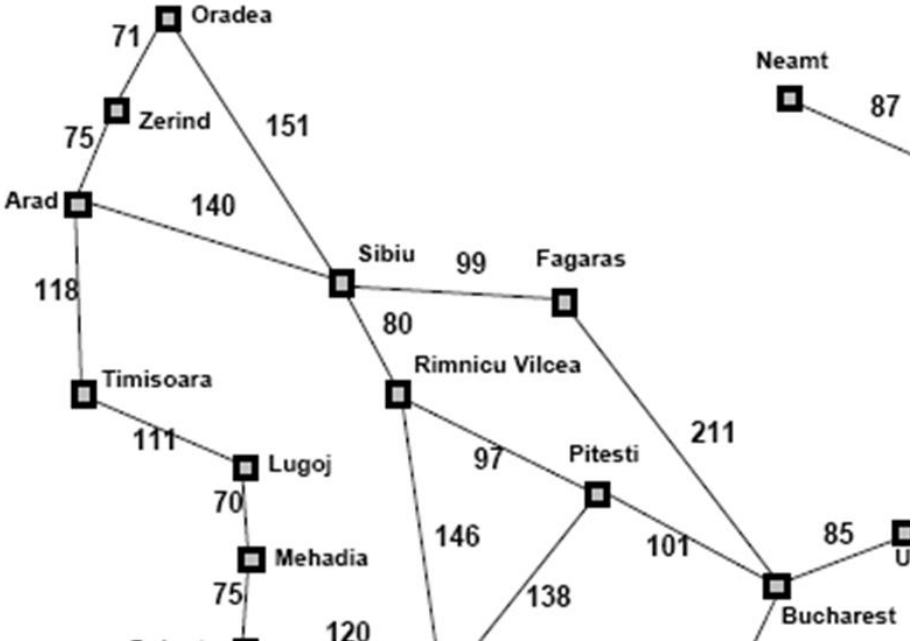
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374



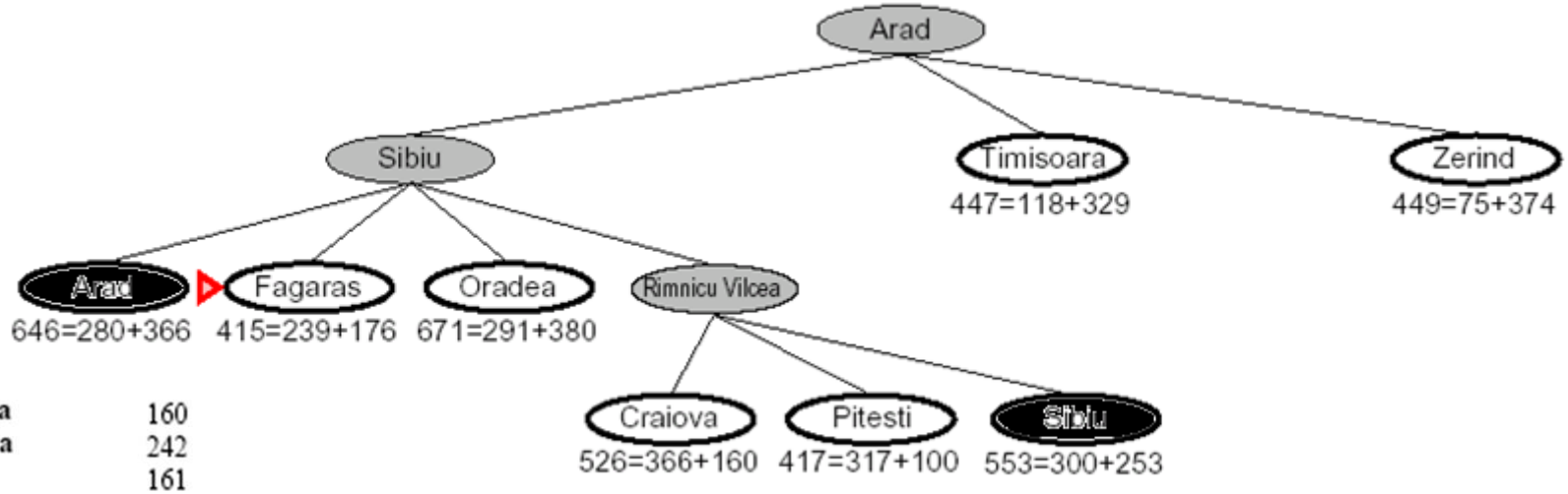
Exemplo: busca A*



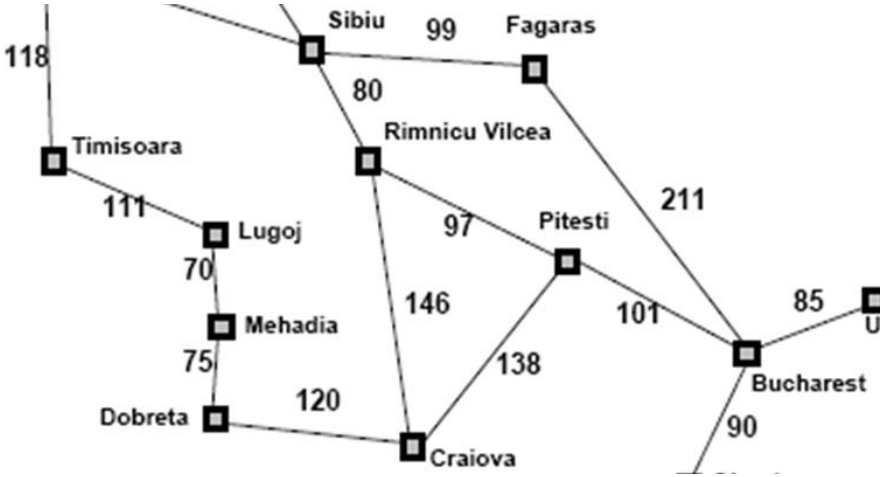
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374



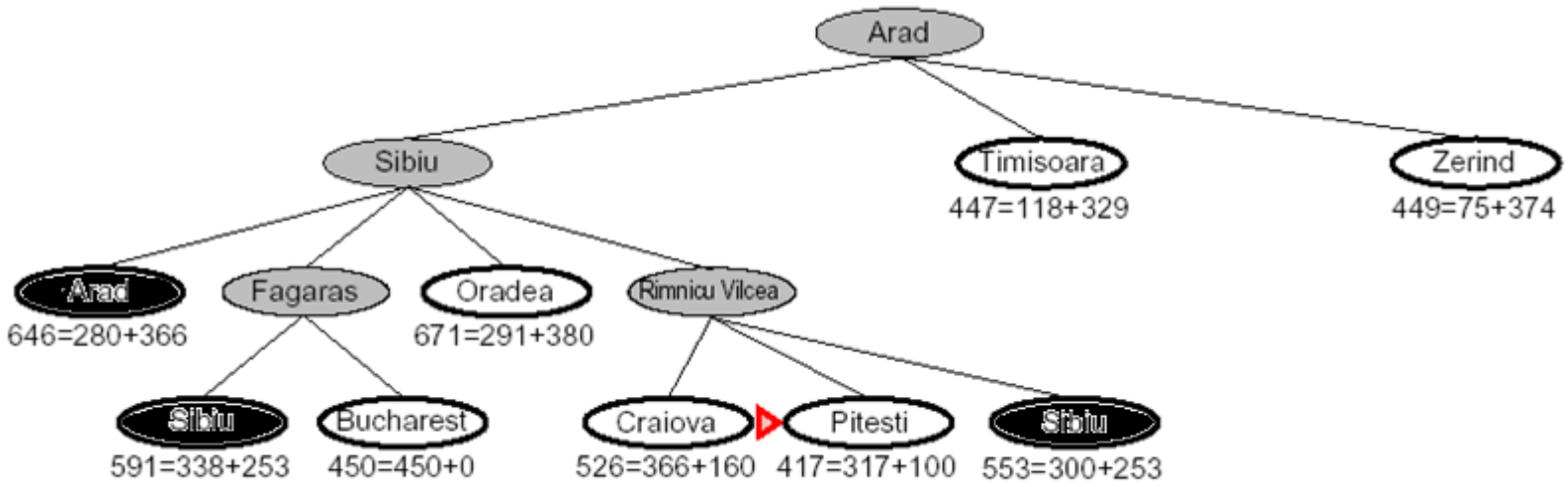
Exemplo: busca A*



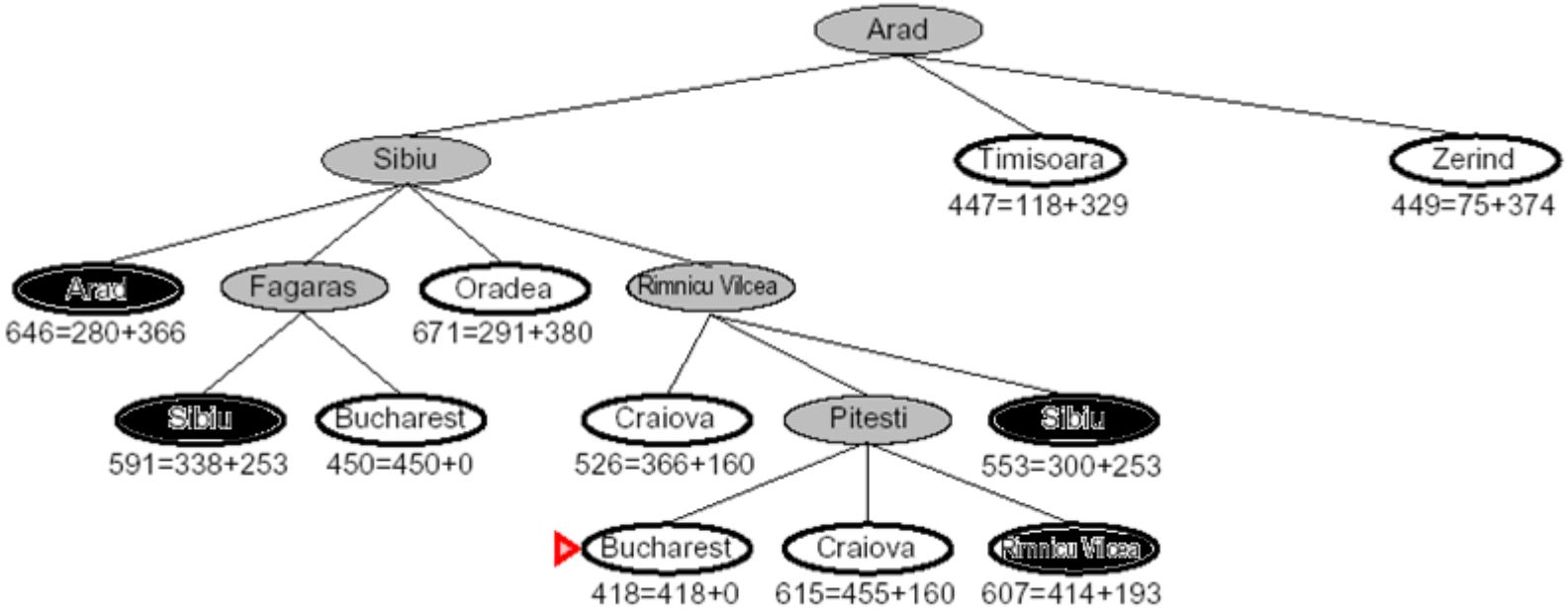
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374



Exemplo: busca A*



Exemplo: busca A*



Algoritmo da busca A*

- A* é uma mistura de busca pelo menor-custo-primeiro (baseada em *cost(n)*) e busca pelo melhor-primeiro (baseada em *h*(*n*)).

- Ela trata a fronteira como uma ***fila de prioridade*** ordenada por ***f*(*p*)**.

- Ela sempre seleciona o nó ***p*** na fronteira com a menor distância estimada a partir do início até um nó objetivo, restringida a passar pelo nó ***p***.

Algoritmo da busca A*


```
def busca_A*(G, S, goal):
   entrada: G # a grafo
            S # um conjunto de nós iniciais
            objetivo # função booleana que testa se S é um nó objetivo

   saída: um caminho de um membro de S para um nó para o qual a função
   objetivo é verdadeira, ou ⊥ se não existir solução.

   Local: fronteira # um conjunto de caminhos


    fronteira ← {<s>|s ∈ S}
    while fronteira :
        selecione e remova o 1º nó <s_0,…,s_k> da fronteira

        if objetivo(s_k): return <s_0,…,s_k>

        adicione(ordem crescente de f(n), fronteira, {<s_0,…,s_k,s>}|<s_k,s> ∈ S})
    return ⊥
```

Análise da busca A*

- Vamos supor que os custos do arco são estritamente positivos.
- Complexidade de tempo é $O(b^m)$
  - Se a heurística for completamente não informativa e os custos dos arcos forem os mesmos, a busca A* faz a mesma coisa que a busca em largura.
- Complexidade de espaço é $O(b^m)$ como na busca em largura, a busca A* mantém uma fronteira que cresce com o tamanho da árvore
- Completa: Sim.
- Ótima: Sim.

Admissibilidade da busca A*

- Se houver uma solução, a busca A* sempre encontra a solução ótima – i.e. o primeiro caminho para um nó objetivo selecionado – se:
  - O fator de ramificação é finito.
  - O custo dos arcos são delimitados acima de zero (existe algum $\boldsymbol{\varepsilon} > 0$ tal que todos os custos dos arcos são superiores a $\boldsymbol{\varepsilon}$).
  - $\boldsymbol{h(n)}$ é uma subavaliação do comprimento do caminho mais curto da $\boldsymbol{n}$ para um nó de objetivo (i.e. uma heurística admissível).
- No entanto, a complexidade de espaço ainda é um problema.

Otimalidade de A*

- *Se $h(n)$ for consistente,*
  - *então os valores de $f(n)$ ao longo de qualquer caminho serão crescentes.*
- Suponha que n' seja sucessor de n, então

$$c(n') = c(n) + c(n, \boldsymbol{a}, n')$$

- para alguma ação $\boldsymbol{a}$ e teremos:

$$f(n') = c(n') + h(n')$$

$$= c(n) + c(n, a, n') + h(n')$$

$$\geq c(n) + h(n) = f(n).$$

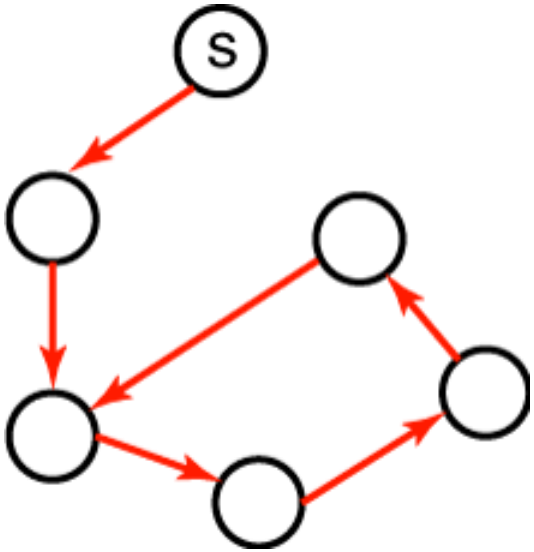
- Assim, o primeiro nó objetivo selecionado para a expansão deve ser uma solução ótima porque
- f é o custo real para nós objetivo (que têm $h = 0$)
- e todos os outros nós objetivo visitados posteriormente terão custo igual ou maior.

$$
\begin{aligned}
f(n') &= c(n') + h(n') \\
&= c(n) + c(n, a, n') + h(n') \\
&\geq c(n) + 0 \\
&= c(n) = f(n).
\end{aligned}
$$

Porque a busca A* é ótima?

- Há sempre um elemento do caminho da solução ótima na fronteira, antes de um nó objetivo ter sido selecionado.
  - Isso ocorre porque, no algoritmo de pesquisa abstrata, existe a parte inicial de cada caminho para um nó objetivo.
- A busca A* pára, pois os custos dos caminhos na fronteira continuam a aumentar e, eventualmente, eles excederão qualquer número finito.
- Se os custos forem positivos não há necessidade de verificar a existência de ciclos na busca A*, pois os custos dos caminhos com ciclos serão sempre maiores e, portanto, não serão escolhidos.

Verificação de Ciclos

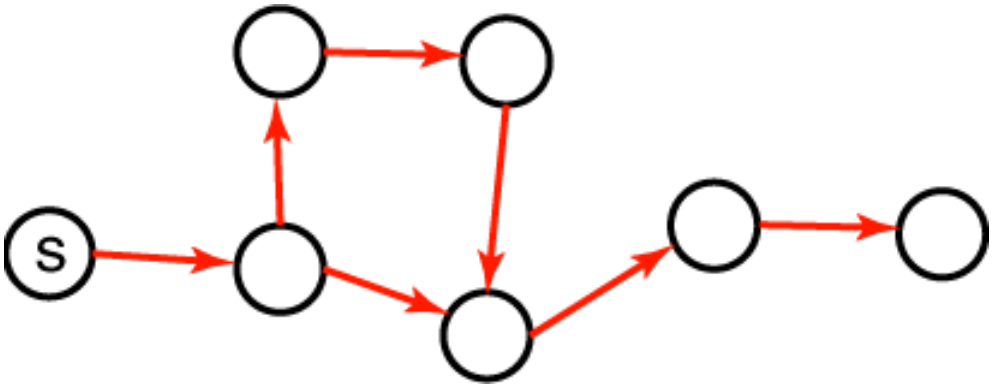


- Um buscador pode podar um caminho que termina em um nó que já está no caminho, sem remover a solução ideal.
  - Usando métodos de busca em profundidade, com o grafo explicitamente armazenado, isso pode ser feito em tempo constante.
  - Para outros métodos, o custo é linear no comprimento do caminho.

Algoritmo genérico de busca em grafo com detecção de ciclo


```
def busca_em_grafo(G, S, goal):
   entrada: G # a grafo
            S # um conjunto de nós iniciais
            objetivo # função booleana que testa se S é um nó objetivo

   saída: um caminho de um membro de S para um nó para o qual a função
   objetivo é verdadeira, ou ⊥ se não existir solução.

   Local: fronteira # um conjunto de caminhos
          fechado # conjunto de nós já visitados

   fechado ← {}
   fronteira ← {<s>|s ∈ S}

   while fronteira ≠ {}:
       seleciona e remove <s_0,…,s_k> da fronteira

       if not (s_k ∈ fechado):
           if objetivo(s_k): return <s_0,…,s_k>
           fechado ← fechado ∪ {s_k}
           fronteira ← fronteira ∪ {<s_o,...,s_k, s>|<s_k,s> ∈ S} # varia com o algoritmo
   return ⊥
```

Remoção de caminhos múltiplos



- A remoção de caminhos múltiplos consiste em podar um caminho para o nó ***n*** para o qual o buscador já tenha encontrado um outro caminho.
- A remoção de caminhos múltiplos engloba a verificação de ciclos.
- Isto implica em armazenar na memória todos os nós para os quais já se encontrou caminhos.
- É importante garantir que após a remoção uma solução ótima ainda pode ser encontrada.

Remoção de caminhos múltiplos e soluções ótimas

- Problema: o que fazer se um caminho subsequente para ***n*** é menor do que o primeiro caminho para ***n***?
  - Remova todos os caminhos da fronteira que usam o caminho mais longo.
  - Altere o segmento inicial dos caminhos na fronteira para usar o caminho mais curto.
  - Certifique-se de que isto não aconteça, garantindo que o caminho mais curto para um nó é encontrado antes.

Remoção de caminhos múltiplos e busca A*

- Suponha que o caminho ***p*** para o nó ***n*** foi selecionado, mas ainda há um caminho mais curto para ***n***. Suponha que esse caminho mais curto é via o nó ***p´*** na fronteira.
- Suponha que o caminho ***p´*** termina no nó ***n´***.
- cost(p) + h(n) ≤ cost($p´$) + h($n´$) porque ***p*** foi selecionado antes de ***p´***.
- cost($p´$) + cost($n´$, n) < cost(p) porque o caminho para ***n*** via ***p´*** é mais curto.

  cost(n´, n) < cost(p) - cost(p´) ≤ h(n´) - h(n):

- Você pode garantir que isso não ocorrerá caso:

  |h(*n´*) - h(*n*)| ≤ cost(*n´*, *n*).

Restrição monotônica

- A função heurística ***h*** satisfaz a restrição monotônica se $|h(m) - h(n)| \leq$ custo (m, n) para cada arco $<m, n>$.
- Se ***h*** satisfaz a restrição monotônica, a busca A* com remoção de caminhos múltiplos sempre encontra o caminho mais curto para um objetivo.
- Este é um reforço ao critério de admissibilidade.

Quando usar?

- Remoção de múltiplos caminhos engloba a checagem de ciclos.
  - Pode ser feito em tempo constante se todos os nós encontrados estiverem armazenados.
- Remoção de múltiplos caminhos.
  - Métodos em largura primeiro – quando temos que armazenar virtualmente todos os nós considerados.
- Checagem de ciclos.
  - Métodos em profundidade primeiro – quando armazenamos somente o caminho para o qual estamos realizando a busca.

Busca recursiva de melhor escolha

- É uma forma de combinar a busca em profundidade com informações de heurísticas.
- Os nós vizinhos (nós gerados) são ordenados antes de serem pesquisados.
  - Seleciona localmente qual subárvore expandir, mas continua fazendo busca em profundidade.
- A complexidade de espaço é linear no tamanho do caminho.
- Não garante encontrar uma solução (como a busca em profundidade), mas se encontrar é a solução ideal.
- Mais útil quando há várias soluções e queremos uma solução ótima.

Nome anterior: Busca em aprofundamento por enumeração e poda

Busca recursiva de melhor escolha

- Ideia: manter o custo do caminho de menor-custo encontrado até um objetivo até agora, chamar isto de ***limite***.

- Se a busca encontrar um caminho ***p*** tal que
  - f(p) = *cost(p)* + h(p) ≤ *limite* o caminho ***p*** pode ser removido.

- Se for encontrado um caminho não-podado para um objetivo, ele deve ser melhor do que o melhor caminho anterior. Esta nova solução é lembrada e o *limite* é definido como o seu custo.

- A busca pode ser uma busca em profundidade para economizar espaço.

- Como o *limite* deve ser inicializado?

Busca recursiva de melhor escolha: Inicializando o *limite*

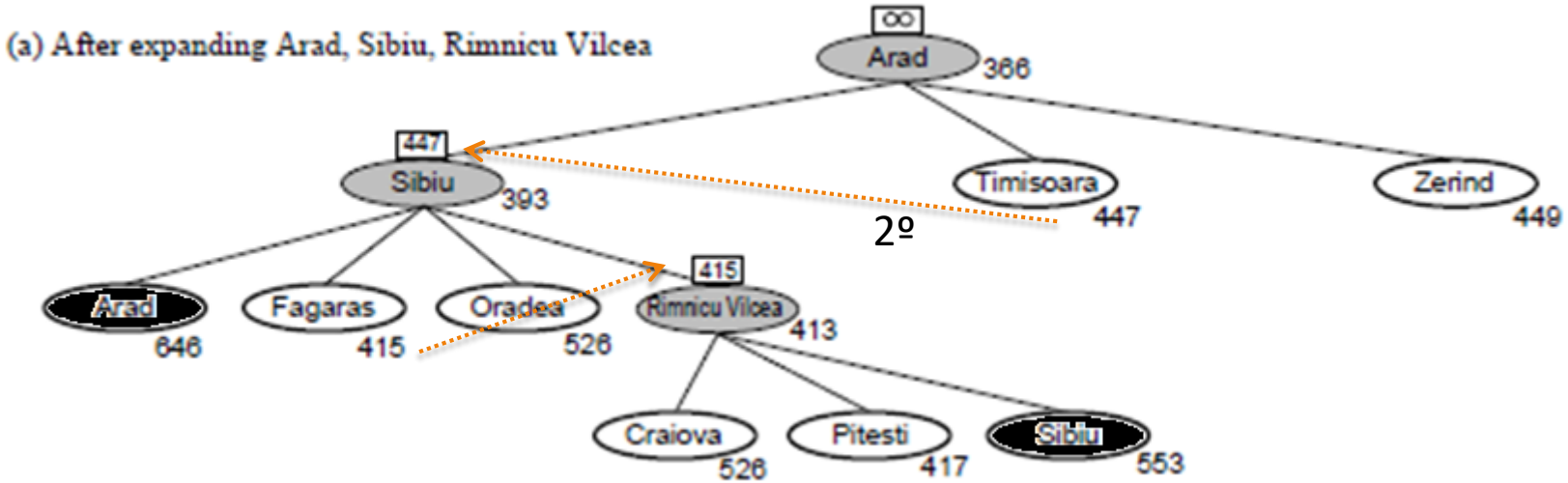
- O *limite* pode ser inicializado em ∞.
- O *limite* pode ser definido como uma estimativa do custo do caminho ótimo. Após a busca em profundidade terminar podemos ter as seguintes situações:
  - Uma solução foi encontrada.
  - Nenhuma solução foi encontrada e nenhum caminho foi podado.
  - Nenhuma solução foi encontrada e um caminho foi removido.

Busca recursiva de melhor escolha

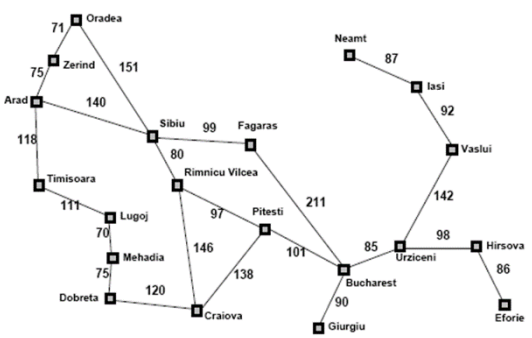
função BUSCA-RECURSIVA-PELA-MELHOR(*problema*) **retorna** uma solução ou falha
  retorna RBFS(*problema*, FAZ-NÓ(*problema*. ESTADO-INICIAL), ∞)

  função RBFS (*problema*, *nó, f_limite*) **retorna (**uma solução ou falha) e um limite novo *f_custo*
    se problema.TESTE-OBJETIVO(nó.ESTADO) **então retorne** SOLUÇÃO (*nó*)
    sucessores ← [ ]
    para cada *ação* **em** *problema*. AÇÕES(nó.ESTADO) **fazer**
      adicionar NÓ-FILHO(*problema, nó, ação*) em *sucessores*
    se *sucessores* estiver vazio **então retornar** falha, ∞
    para cada *s* **em** *sucessores* **fazer** / * atualizar *f* com o valor da busca anterior, se houver */
      s.*f* ← max(*s.g* + *s.h, nó.f*) /* s.g é o custo do caminho acumulado */
    repita
      melhor ← seleciona nó com valor *f* mais baixo em *sucessores*
      se *melhor.f* > *f_limite* **então retornar** *falha, melhor.f*
      alternativa ← nó com segundo valor *f* mais baixo entre *sucessores*
      resultado, *melhor.f* ← RBFS(*problema, melhor*, min(*f_limite*, *alternativa*)
      se *resultado* ≠ *falha* **então retornar** *resultado*

Exemplo: Busca recursiva de melhor escolha

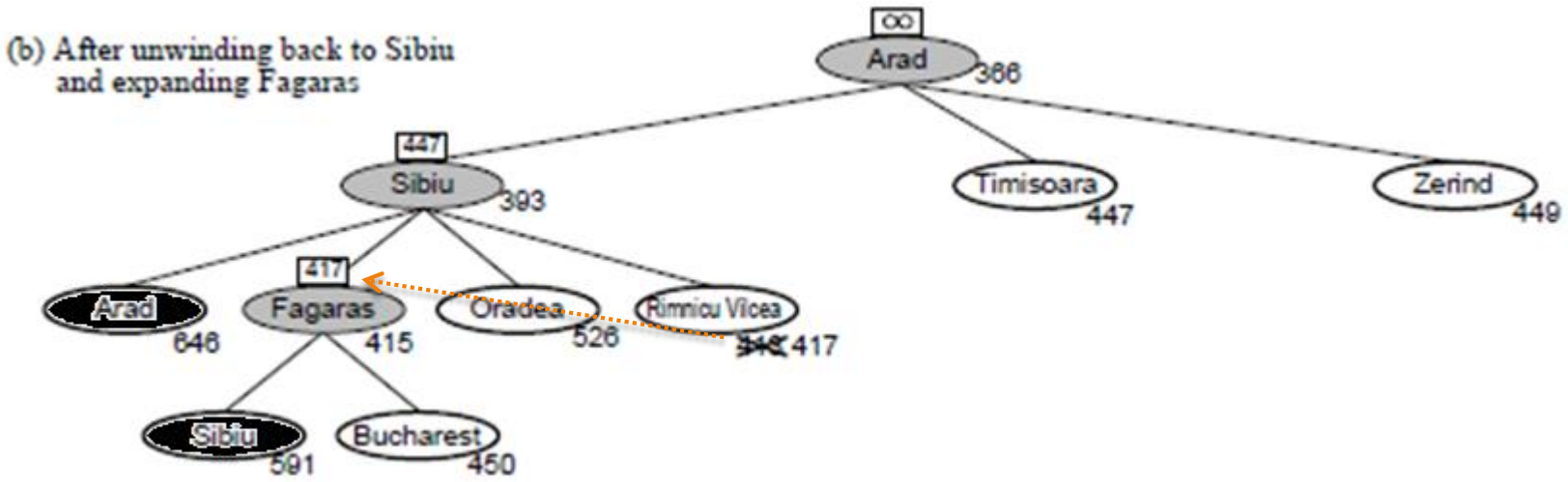


417 de Pitesti é pior do que 415 do topo de Rimnicu Vicea
Portanto será retrocedido
Rimnicu Vicea será atualizado com f(n) de Pitesti = 417



Straight−line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Exemplo: Busca recursiva de melhor escolha

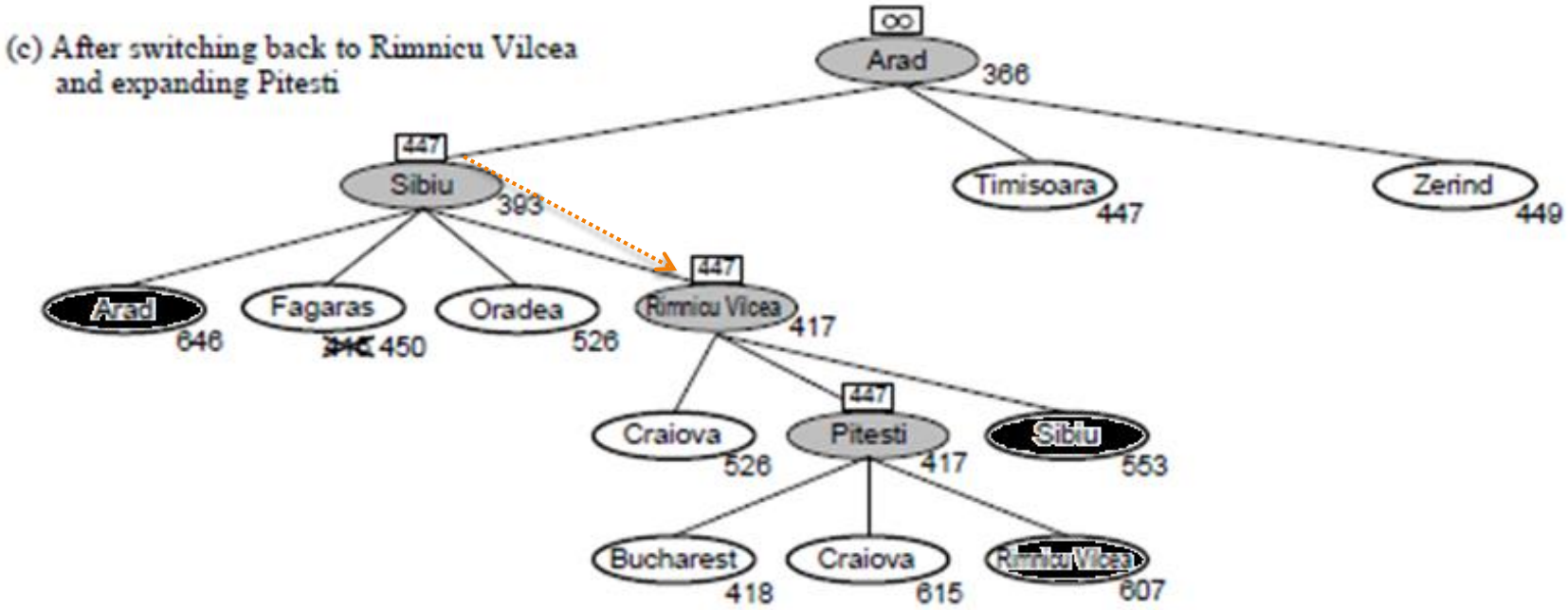


Fagaras é escolhida novamente pois seu custo f(n) = 415 é menor do que de Rinicu Vicea (f(n) = 417)

No entanto, Bucharest (f(n) = 450) supera o custo de Rimnicu Vicea (f(n) = 417)

Então Rimnicu Vicea é escolhida novamente

Exemplo: Busca recursiva de melhor escolha



Neste caso como o custo de Fagaras (f(n) = 450) supera o custo do pai Sibiu (f(n) = 447)

Então no topo de Rimnicu Vicea é assumido f(n) = 447

E retorna a solução Bucharest (pois 418 é menor do que o limite 447 de Pitesti)

Busca recursiva de melhor escolha

- Completa: não, pelas mesmas razões que a busca em profundidade não é completa.
  - No entanto, para muitos problemas de interesse não há nenhum caminho infinito e nenhum ciclo, portanto, para muitos problemas esta busca é completa.
- Complexidade de tempo: **$O(b^m)$**
- Complexidade de espaço: **$O(bm)$**
  - Enumeração e poda tem a mesma complexidade de espaço da busca em profundidade.
- Ótima: Sim

Sumário: Busca

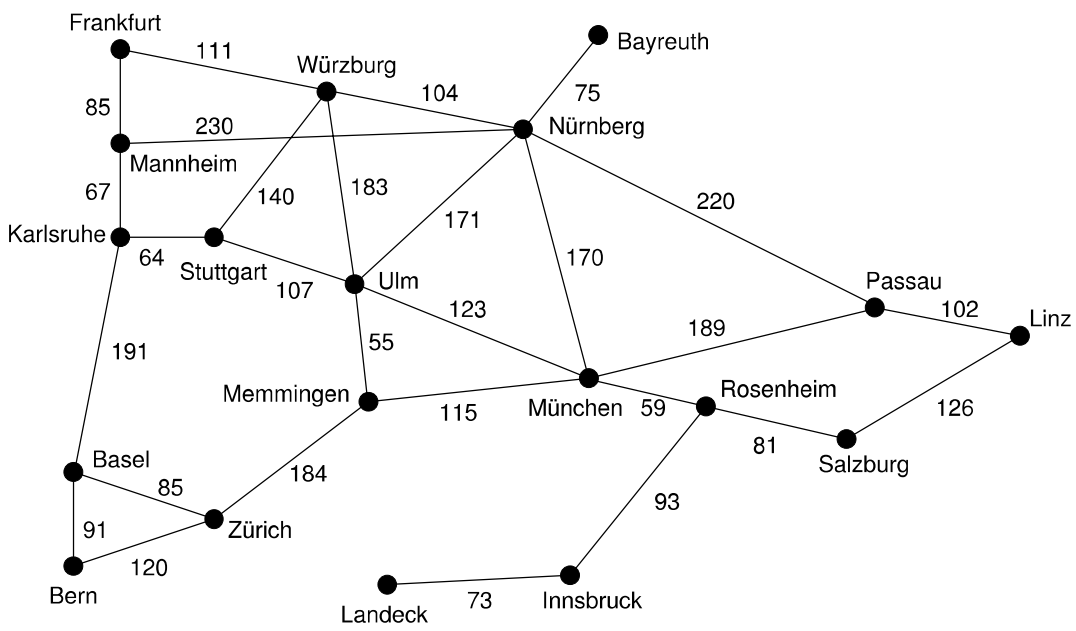
Tipo de Busca	Seleção da fronteira	Completa	Ótima	Custo de Tempo	Custo de Espaço
Largura	Primeiro nó adicionado	S	S → custos constantes	$O(b^m)$	$O(b^m)$
Profundidade	Último nó adicionado	N S → se espaço de busca for sem ciclos e finito	N	∞ ou $O(b^m)$	$O(mb)$
Custo Uniforme	Nó de custo mínimo → *cost*(n)	S → custos > 0	S → custos > 0	$O(b^m)$	$O(b^m)$
Aprofundamento interativo	Último nó adicionado	N S → se espaço de busca for sem ciclos e finito	S → custos constantes	$O(b^m)$	$O(mb)$
Melhor-primeiro	Nó de custo mínimo local → *h*(n)	N	N	$O(b^m)$	$O(b^m)$
A*	Nó de custo mínimo estimado → *f*(n)	S	S	$O(b^m)$	$O(b^m)$
Busca recursiva de melhor escolha	Nó de custo mínimo estimado → *f*(n)	N S → se espaço de busca for sem ciclos e finito	S	∞ ou $O(b^m)$	$O(mb)$

Simulações da expansão de busca

Busca em largura bi-direcional

Exercícios:

- Dado o mapa abaixo aplique os algoritmos discutidos com o estado inicial em **Frankfurt** e o estado final em **Ulm**.



Basel	204
Bayreuth	207
Bern	247
Frankfurt	215
Innsbruck	163
Karlsruhe	137
Landeck	143
Linz	318
München	120
Mannheim	164
Memmingen	47
Nürnberg	132
Passau	257
Rosenheim	168
Stuttgart	75
Salzburg	236
Würzburg	153
Zürich	157

Bibliografia

- Poole, D. L., & Mackworth, A. K. (2010). *Artificial Intelligence: foundations of computational agents*. Cambridge University Press.
- Russell, S., & Norvig, P. (2013). Inteligência Artificial: uma abordagem moderna. Terceira edição. GEN LTC.